

Telemetry Protocol Specification

The AirOne wire format is a compact binary frame with a fixed header, a variable payload, an optional 8-byte authentication tag, and a trailing CRC32. It is implemented in `src/telemetry/protocol.py`.

Frame layout

Offset	Size	Field
-----	----	-----
0	4	MAGIC = A1 60 4E 45
4	1	VERSION = 0x71
5	1	PACKET_TYPE
6	4	SEQUENCE_NUMBER (uint32, little-endian)
10	8	TIMESTAMP (uint64, little-endian)
18	2	PAYLOAD_LENGTH N (uint16, little-endian)
20	1	FLAGS (bit0=FEC_PRESENT, bit1=COMPRESSED, bit2=ENCRYPTED, bit3=AUTHENTICATED)
21	N	PAYLOAD
21+N	8	AUTH_TAG – only when FLAGS bit3 is set
21+N+T	4	CRC32 over every preceding byte (header, payload, tag) (uint32, LE)

`PAYLOAD_LENGTH` counts the payload only (never the tag). Unknown `VERSION`, `PACKET_TYPE` or `FLAGS` bits cause the frame to be rejected and counted (`invalid_version` / `invalid_type` / `invalid_flags`) — never parsed.

- Header size: **21 bytes**. CRC size: **4 bytes**.
- Header struct format: `<4sBBIQHB`.
- Maximum payload: **65535 bytes** (bounded by the uint16 length field).

Constants

Name	Value
MAGIC	A1 60 4E 45
VERSION	0x71
HEADER_SIZE	21
CRC_SIZE	4
MAX_PAYLOAD	65535
FLAG_FEC_PRESENT	0x01
FLAG_COMPRESSED	0x02
FLAG_ENCRYPTED	0x04
FLAG_AUTHENTICATED	0x08
AUTH_TAG_SIZE	8
MIN_LINK_KEY_BYTES	16

Packet types (PacketType)

Value	Name
0x01	SENSOR_DATA
0x02	GPS
0x03	SYSTEM_STATUS
0x04	COMMAND
0x05	ACK
0x06	HEARTBEAT
0x07	FEC_DATA
0x08	ERROR
0x10	COMPACT (radio link only, see below — never reaches the ground station)

Compact radio payload (PACKET_TYPE 0x10)

One SX1268 LoRa packet holds at most 255 bytes. The JSON `SENSOR_DATA` payload is about 2.5 kB, so it can't be sent over the air as-is. The CanSat therefore sends the **same measurements** as a fixed-schema binary payload (`airone_compact.h`), wrapped in an ordinary AirOne frame: same header, sequence, timestamp, optional HMAC tag and CRC32. The ground bridge (`airone_ground_bridge/`) verifies the frame and expands it back into JSON. It adds the ground-measured `radio_rssi` / `radio_snr` and re-emits it over USB as a normal `0x01 SENSOR_DATA` frame with the same sequence/timestamp, re-signed when a key is configured. The ground station's parser therefore needs no change and continues to reject `0x10` as an unknown type if it ever sees one.

Offset	Size	Field (little-endian)
0	1	schema version (1)
1	1	flags: bit0 = altitude_rel / vertical_speed come from BME688 (else BM-P581)
2	8	presence mask: bit i set = field i present
10	4*k	one 4-byte value per present field, in index order: float32 int32 = deg*1e7 (gnss_lat/lon) uint32 (counters/enums/masks)

There are 39 fields (index order and units in `AC_FIELDS`). Fields absent from the mask are **absent** from the expanded JSON. Nothing is filled in. A full payload is 166 B, and the whole authenticated frame is 199 B. The full 2 Hz JSON frames are kept on board (MicroSD, with W25Q128 failover).

CRC32

The CRC32 is computed with `binascii.crc32` masked to 32 bits and packed little-endian. It covers **every byte preceding the CRC field** — the full header and the payload. A frame whose recomputed CRC does not match is marked `crc_valid = False` ; downstream, the affected data is tagged `CORRUPTED` and is never treated as a real reading.

Frame authentication (FLAG_AUTHENTICATED)

`AUTH_TAG = HMAC-SHA256(link_key, header || payload)[0:8]` . The key (≥ 16 bytes, hex) is `AIRONE_LINK_KEY` on the ground station and `AIRONE_LINK_KEY_HEX` in the firmware; `compute_auth_tag()` / `airone_auth_tag()` are byte-for-byte identical (tested against RFC 4231). The tag is compared in constant time. The resulting `auth_state` is explicit:

auth_state	Frame has tag	Ground station has key	Result
NOT_CONFIGURED	no	no	accepted as before
AUTHENTICATED	yes	yes, matches	accepted
UNAUTHENTICATED	no	yes	SUSPECT ; INVALID when telemetry.require_authenticated_frames is true
UNVERIFIABLE	yes	no	SUSPECT — cannot be checked
INVALID_TAG	yes	yes, mismatch	AuthTagError ; emitted as a corrupted packet with no measurements

The tag provides integrity and origin authentication only — the payload is **not** encrypted (`FLAG_ENCRYPTED` remains reserved). Replay is handled by sequence tracking: a sequence far behind the newest one is flagged `replay = True` and counted.

Stream parsing

`StreamParser.parse_stream(data: bytes) -> List[ParsedPacket]` is resync-safe:

- It scans for the `MAGIC` sync word, so partial or noisy streams recover at the next valid frame boundary rather than desynchronising permanently.
- Each `ParsedPacket` exposes `.header` (dict), `.payload` (bytes), `.crc_valid`, `.duplicate`, `.out_of_order`, `.replay`, `.auth_state`, `.rejected`, and a `.sequence` property. `StreamParser.counters()` exposes all diagnostic counters.
- Duplicate and out-of-order frames are **flagged, not silently dropped**, so the quality accounting downstream stays honest.

Forward Error Correction (optional)

When `FLAG_FEC_PRESENT` is set and the `reedsolo` package is installed, the pipeline's FEC stage attempts Reed-Solomon recovery. If `reedsolo` is **not** installed, FEC is disabled explicitly and reported — corrupted frames are marked `CORRUPTED` rather than silently passed through as if valid.

Decoding to measurements

`decode_payload_to_frame(pkt)` converts a `ParsedPacket` into a `TelemetryFrame` of fully-provenanced `Measurement` objects. Each measurement carries its value, unit, timestamp, sensor id, `QualityState`, `DataSource`, and uncertainty — never a bare number.

Payload limits (hostile input never raises): UTF-8 JSON object, ≤ 16 KiB, ≤ 64 fields, nesting depth ≤ 3 , strings ≤ 64 chars. Non-finite or non-numeric values become `INVALID`; a declared `source` may only downgrade provenance to `SIMULATED`; implausible timestamps are replaced by receive time and flagged `timestamp_replaced`.